
Teoria da Informação e o TF-IDF

Aula 1,5 — De Shannon aos pesos dos termos

Projeto Integrador III ■ Ciência de Dados

FATEC — Tecnologia em Ciência de Dados

Um jogo

Estou pensando em **um** dos 8 documentos do nosso *corpus*. Você precisa descobrir qual, fazendo apenas perguntas de **sim ou não**.

- Quantas perguntas você precisa, no pior caso?
- Se eu disser “o documento contém a palavra *busca*”, quanto isso **ajuda**?
- E se eu disser “contém a palavra *de*”? Ajuda **o mesmo tanto**?

Responder isso com precisão é o que fará o TF-IDF deixar de ser fórmula decorada.

Relembrando: o nosso *corpus*

```
docs <- c(  
  d1 = "recuperacao de informacao ordena documentos por relevancia",  
  d2 = "o modelo de espaco vetorial representa documentos como vetores",  
  d3 = "bm25 e um modelo probabilistico de ranqueamento de texto",  
  d4 = "aprendizado estatistico fundamenta a recuperacao moderna",  
  d5 = "o indice invertido acelera a busca em muitos documentos",  
  d6 = "embeddings capturam a semantica de palavras e documentos",  
  d7 = "a avaliacao mede a relevancia dos resultados da busca",  
  d8 = "ciencia de dados combina estatistica e programacao"  
)  
length(docs)  
  
[1] 8
```

Oito documentos curtos. É sobre eles que vamos medir **informação**.

O problema fundamental da comunicação

“Reproduzir num ponto, exata ou aproximadamente, uma mensagem **selecionada** em outro ponto.”

- Existe um **conjunto** de mensagens possíveis, conhecido pelos dois lados.
- O emissor **seleciona uma**.
- Informação é o que permite ao receptor descobrir **qual**.

Informação não é *conteúdo*: é **escolha entre alternativas**.

Passo 0: reescrever a busca como problema de seleção

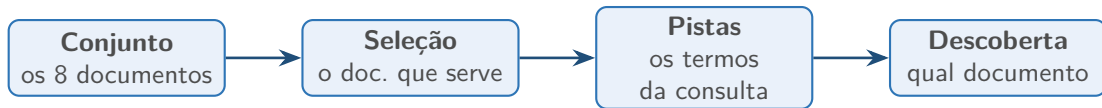
Shannon exige duas coisas: um **conjunto** de mensagens possíveis e uma **escolha** entre elas. Traduzindo a busca para esse formato:

- **O conjunto:** os N documentos do *corpus*.
- **A escolha:** existe um documento (ou alguns) que atende à necessidade do usuário.
- **O canal:** a consulta são **pistas** sobre qual foi escolhido.

Reformulada assim

A busca vira **exatamente** o problema de Shannon: identificar o item selecionado usando o **mínimo de evidência**.

O mesmo problema, em diagrama

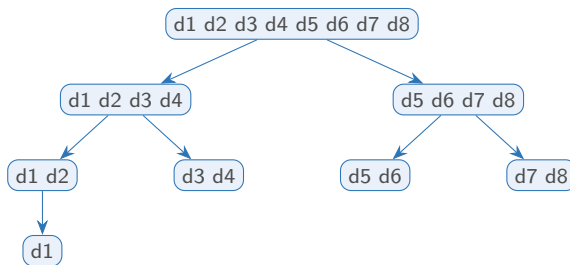


- Note o que **não** entra: se o documento diz a **verdade**, se é bem escrito, se é útil.
- Só entra o que permite **distinguir** um documento dos outros.

Cada termo da consulta estreita o conjunto. O passo 1 mede *o quanto*.

Passo 1: quanta incerteza existe no início?

Com 8 documentos igualmente prováveis, quantas perguntas de sim/não bastam?



Cada pergunta corta o conjunto **pela metade**: $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$. São **3** perguntas.

O bit: a unidade de uma pergunta

Definição

1 bit é a informação de *uma* resposta sim/não com as duas opções igualmente prováveis.

```
N <- length(docs)
log2(N)           # perguntas necessarias para achar 1 entre N

[1] 3
```

O logaritmo aparece porque ele **conta quantas vezes** dá para dividir N por 2. Se fossem 1024 documentos, seriam apenas $\log_2 1024 = 10$ perguntas.

Uma pista vale quantos bits?

Eu digo: “o documento contém a palavra *busca*”.

```
tok      <- function(x) unlist(strsplit(x, " "))
contem   <- function(t) names(docs)[sapply(docs, function(d) t %in% tok(d))]
contem("busca")

[1] "d5" "d7"

log2(8 / 2)      # de 8 candidatos passei para 2

[1] 2
```

Sobraram **2** candidatos: a pista pagou **2** dos 3 bits. Resta 1 bit de incerteza.

Generalizando: a informação de uma pista

Se o termo aparece em df dos N documentos, a probabilidade de um documento sorteado contê-lo é

$$P(t) = \frac{df_t}{N}$$

e a pista reduz o conjunto de N para df , custando

$$I(t) = \log_2 \frac{N}{df_t} = -\log_2 P(t)$$

Essa quantidade tem nome: **autoinformação** (ou *surprisal*). Guarde a fórmula — já vamos reencontrá-la.

Por que *logaritmo*? Três exigências

Antes de escolher a fórmula, decida o que ela precisa cumprir:

- ① **Evento certo não informa nada:** $I(1) = 0$.
“O sol nasceu hoje” não acrescenta nada.
- ② **Quanto mais raro, mais informa:** I decrescente em p .
- ③ **Informação é aditiva:** para eventos independentes,
 $I(p_1 p_2) = I(p_1) + I(p_2)$.

A consequência

A exigência 3 pede uma função que transforme **produto em soma** — e só o logaritmo faz isso. Com o sinal para ficar decrescente: $I(p) = -\log p$.

E se fosse multiplicativa?

Suponha $I(p_1 p_2) = I(p_1) \cdot I(p_2)$. Faça $p_1 = p_2 = 1$:

$$I(1) = I(1) \cdot I(1) \implies I(1) = I(1)^2 \implies I(1) \in \{0, 1\}$$

- Se $I(1) = 0$, então $I(p) = I(p) \cdot I(1) = 0$ **para todo** p : função nula, inútil.
- Se $I(1) = 1$, o evento **certo** carregaria informação — contradiz a exigência 1.

Em uma frase

Somar tem **neutro 0**; multiplicar tem **neutro 1**. Só o 0 pode significar “nenhuma informação”.

A fórmula em números

```
I <- function(p) -log2(p)
I(c(1, 1/2, 1/4, 1/8))      # certo, moeda, 2 moedas, 3 moedas

[1] 0 1 2 3
```

Evento	P	$I = -\log_2 P$
o sol nasceu	1,000	0,00 bits
cara numa moeda	0,500	1,00 bit
face 6 de um dado	0,167	2,58 bits
termo em 1 de 8 docs	0,125	3,00 bits
palavra rara (1 em mil)	0,001	9,97 bits

O que os bits *não* medem

Shannon foi explícito no artigo de 1948:

“Frequentemente as mensagens têm **significado**. . . Esses aspectos semânticos são **irrelevantes** para o problema de engenharia.”

- “Está chovendo” e “sua casa pegou fogo” podem ter **os mesmos bits**.
- Uma mensagem **falsa** tem a mesma informação que uma verdadeira, se tão provável quanto.
- Bits medem **incerteza resolvida**, não importância, utilidade ou verdade.

Guarde isso: **informação** $\neq$ **relevância**. Voltaremos a esse ponto na Aula 05.

A ponte: isso já é o IDF

Lembre da Aula 01. Definimos o peso de raridade de um termo como

$$\text{IDF}(t) = \log \frac{N}{\text{df}_t}$$

e acabamos de derivar, **sem olhar para o TF-IDF**, que a informação de uma pista é

$$I(t) = \log \frac{N}{\text{df}_t}$$

São a mesma coisa

O **IDF** não é uma heurística: ele é a **autoinformação** do termo. Mede quantos bits aquele termo paga da dívida de $\log_2 N$.

Conferindo no nosso *corpus*

```
df <- c(modelo = 2, de = 5, recuperacao = 2)
round(log2(8 / df), 3)      # IDF em BITS
```

modelo	de	recuperacao
2.000	0.678	2.000

- modelo e recuperacao (2 docs cada): **2 bits** — cada um resolve dois terços do problema.
- de (5 docs): **0,68 bits** — quase não estreita o conjunto.

No código da Aula 01 usamos `log()`, que em R é o logaritmo **natural**: os mesmos IDF's em *nats* (1,39 e 0,47). A base muda a unidade, **não muda o ranking**.

Falta o TF: e se o termo aparecer várias vezes?

O IDF trata todos os documentos que contêm o termo **como iguais**. Mas um documento que repete “praia” cinco vezes parece falar mais do assunto.

Modelo mais simples possível

Trate cada ocorrência como uma **observação independente** da mesma pista. Pela **aditividade** (exigência 3), tf observações valem tf vezes a informação:

$$tf_{t,d} \times I(t)$$

E acabamos de reconstruir o TF-IDF — desta vez **deduzido**, não decorado.

$$\text{tfidf}(t, d) = \underbrace{\text{tf}_{t,d}}_{\text{quantas observações}} \times \underbrace{\log \frac{N}{\text{df}_t}}_{\text{bits por observação}}$$

Nova leitura

Cada célula da matriz TF-IDF responde: “**quantos bits de evidência este termo fornece a favor deste documento?**”

O escore de um documento é a **soma** dos bits que a consulta acumulou nele.

Exemplo completo: a consulta em bits

Consulta: "modelo de recuperacao". IDFs: modelo= 2,00; de= 0,68; recuperacao= 2,00 bits.

contribuicao de cada termo (tf x idf, em bits) e o total por documento

doc	modelo	de	recuperacao	total
d3	2,00	1,36	0,00	3,36
d1	0,00	0,68	2,00	2,68
d2	2,00	0,68	0,00	2,68
d4	0,00	0,00	2,00	2,00
d5	0,00	0,00	0,00	0,00

Em d3, de vale 1,36 porque aparece **duas vezes** ($2 \times 0,68$). E d5 recebe **zero bits**: a consulta não disse nada sobre ele.

Quando a conta fecha certinho

Duas pistas independentes devem **somar** exatamente os 3 bits necessários.

```
intersect(contem("busca"), contem("documentos"))
```

```
[1] "d5"
```

```
log2(8/2) + log2(8/4)      # 2 bits + 1 bit
```

```
[1] 3
```

"busca" (2 docs) mais "documentos" (4 docs) identificam **um único** documento — e os bits somam **3**, exatamente o custo de achar 1 entre 8. A aditividade funcionou.

Quando a conta *não* fecha

```
intersect(contem("recuperacao"), contem("relevancia"))
```

```
[1] "d1"
```

```
log2(8/2) + log2(8/2)      # 2 bits + 2 bits
```

```
[1] 4
```

Também sobrou **um** documento — mas a soma deu **4** bits, e achar 1 entre 8 custa só **3**. Pagamos um bit a mais.

Por quê?

Os termos **não** são independentes: “recuperação” e “relevância” são do mesmo assunto e co-ocorrem mais que o acaso. A segunda pista já era **parcialmente esperada**, logo trouxe menos surpresa que seus 2 bits nominais.

Um caso extremo: e se todos usarem as *mesmas* palavras?

Oito **rotas** de entrega pela Baixada Santista. Todas passam pelas mesmas quatro cidades — muda apenas a **ordem** do percurso.

```
rotas <- c(  
  r1 = "santos cubatao guaruja bertioga",  
  r2 = "santos guaruja cubatao bertioga",  
  r3 = "santos bertioga guaruja cubatao",  
  r4 = "cubatao santos bertioga guaruja",  
  r5 = "guaruja bertioga santos cubatao",  
  r6 = "bertioga cubatao santos guaruja",  
  r7 = "guaruja santos cubatao bertioga",  
  r8 = "bertioga guaruja cubatao santos"  
)
```

Para quem dirige o caminhão, são **oito rotas diferentes**. Vejamos o que o motor de busca enxerga.

A matriz colapsa

```
tokens <- lapply(rotas, function(x) unlist(strsplit(x, " ")))
vocab <- sort(unique(unlist(tokens)))
tdm <- sapply(tokens, function(t) as.integer(table(factor(t, levels = vocab))))
rownames(tdm) <- vocab
tdm
```

	r1	r2	r3	r4	r5	r6	r7	r8
bertioga	1	1	1	1	1	1	1	1
cubatao	1	1	1	1	1	1	1	1
guaruja	1	1	1	1	1	1	1	1
santos	1	1	1	1	1	1	1	1

Todas as células valem **1**: cada cidade aparece uma vez em cada rota.

Zero bits de informação

```
df <- rowSums(tdm > 0)      # cada termo aparece em QUANTOS documentos?
```

```
idf <- log(8 / df)
```

```
df
```

bertioga	cubatao	guaruja	santos
8	8	8	8

```
idf
```

bertioga	cubatao	guaruja	santos
0	0	0	0

O que isso significa

Todo termo tem $df = 8$, logo $IDF = \log(8/8) = 0$. **Toda** a matriz TF-IDF é zero: nenhuma palavra distingue nenhuma rota.

O motor fica cego

A conta da informação

Necessário para achar 1 entre 8: **3 bits**.

Disponível no vocabulário: **0 bits**.

A busca é **impossível**.

No espaço vetorial

Todos os vetores são idênticos:

$$\cos(r_1, r_8) = 1,00$$

O modelo declara as 8 rotas **iguais**.

A lição

Informação exige **contraste**. Sem variação entre documentos, não há o que medir — e aqui a diferença real está na **ordem**, que o saco de palavras **joga fora**.

A terceira mentira: a ordem

- “o cão mordeu o homem” e “o homem mordeu o cão” têm **TF-IDF idêntico**.
- Mesmas palavras, mesmas frequências — sentidos **opostos**.
- O modelo de saco de palavras (*bag of words*) descarta a sintaxe por construção.

Como o curso responde

N-gramas (Aula 03) recuperam vizinhança; **embeddings contextuais** e **cross-encoders** (Aulas 07–08) leem a sequência inteira.

O caso das rotas é artificial de propósito — mas expõe uma limitação que existe em *todo* texto real.

As duas mentiras do TF-IDF

A derivação chegou à fórmula — e revelou onde ela **mente**:

Mentira 1: repetição

A 5ª ocorrência não informa como a 1ª.

Correção: saturação do BM25 (Aula 04).

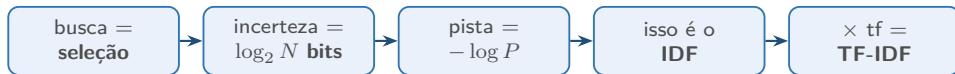
Mentira 2: independência

Termos do mesmo assunto se repetem.

Correção: embeddings (Aula 07) e cross-encoders (Aula 08).

E o limite de escopo

Bits medem **distinguibilidade**, não **utilidade**. Um termo pode ser raro por ser um erro de digitação. Daí a avaliação humana (Aula 05) e o *learning to rank* (Aula 09).



- O logaritmo não é truque: vem da exigência de que informação seja **aditiva**.
- O IDF não é heurística: é **autoinformação** medida em bits.
- O escore de um documento é **evidência acumulada**, em bits.

Parte 1 — explicar e explorar

Para cada bloco de código: **explique** o que faz, **altere** algo e registre o efeito, e **preveja** o resultado antes de rodar.

Parte 2 — investigar

- 1 Se o *corpus* tivesse **1024** documentos, quantos bits custaria achar um? E se dobrasse para 2048?
- 2 Encontre no *corpus* um par de termos cuja soma de bits **superestime** a informação real (como fizemos com recuperacao/relevancia). Explique a correlação.
- 3 Troque $\log_2$ por $\log$ no cálculo do IDF. O **ranking** muda? Por quê?
- 4 Um termo aparece em **todos** os documentos. Quantos bits ele vale? O que isso diz sobre *stopwords*?

Obrigado!

Dúvidas? Vamos ao R.
